```python
import torch.nn as nn
import torch
#from DL_CSILSTMNet import CSILSTMNet

# Define MobileNetV3 block (simplified version)
class MobileNetV3Block1D(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size=3, stride=1, exp_factor=4, se_ratio=0.25):
        super().__init__()
        mid_channels = in_channels * exp_factor
        self.use_residual = (stride == 1 and in_channels == out_channels)

        self.expand_conv = nn.Conv1d(in_channels, mid_channels, kernel_size=1, bias=False)
        self.expand_bn = nn.BatchNorm1d(mid_channels)
        self.expand_act = nn.SiLU()

        self.depthwise_conv = nn.Conv1d(mid_channels, mid_channels, kernel_size, stride, padding=kernel_size//2,
                                        groups=mid_channels, bias=False)
        self.depthwise_bn = nn.BatchNorm1d(mid_channels)
        self.depthwise_act = nn.SiLU()

        se_channels = max(1, int(mid_channels * se_ratio))
        self.se = nn.Sequential(
            nn.AdaptiveAvgPool1d(1),
            nn.Conv1d(mid_channels, se_channels, 1),
            nn.SiLU(),
            nn.Conv1d(se_channels, mid_channels, 1),
            nn.Hardsigmoid()
        )

        self.project_conv = nn.Conv1d(mid_channels, out_channels, 1, bias=False)
        self.project_bn = nn.BatchNorm1d(out_channels)

    def forward(self, x):
        out = self.expand_act(self.expand_bn(self.expand_conv(x)))
        out = self.depthwise_act(self.depthwise_bn(self.depthwise_conv(out)))
        se_weight = self.se(out)
        out = out * se_weight
        out = self.project_bn(self.project_conv(out))

        if self.use_residual:
            return x + out
        return out


class MobileNetV3_1D_LSTM(nn.Module):
    def __init__(self, csi_channels, meta_feature_dim, num_classes):
        super().__init__()
        self.block1 = MobileNetV3Block1D(csi_channels, 32, stride=1)
        self.block2 = MobileNetV3Block1D(32, 64, stride=2)
        self.global_pool = nn.AdaptiveAvgPool1d(1)

        self.lstm = nn.LSTM(meta_feature_dim, 64, 2, batch_first=True, bidirectional=True)
        self.fc = nn.Linear(64 + 64*2, num_classes)

    def forward(self, csi_seq, meta_seq):
        x = csi_seq.permute(0, 2, 1)
        x = self.block1(x)
        x = self.block2(x)
        x = self.global_pool(x).squeeze(-1)

        _, (h_n, _) = self.lstm(meta_seq)
        h_n = torch.cat([h_n[-2], h_n[-1]], dim=1)

        combined = torch.cat([x, h_n], dim=1)
        return self.fc(combined)
```